PRACTICAL – 6

Implementation of chain matrix multiplication using dynamic programming.

Aim:

To implement the **Matrix Chain Multiplication problem using Dynamic Programming** and determine the optimal order of multiplying a sequence of matrices so that the total number of scalar multiplications is minimized.

Objective:
The objective of this experiment is to understand and implement the **Matrix Chain Multiplication problem using Dynamic Programming**. It aims to find the most efficient order of multiplying a given sequence of matrices by minimizing the total number of scalar multiplications required. The experiment also helps in understanding the concepts of optimal substructure and overlapping subproblems in dynamic programming.

Algorithm:

- Start.
- Read the number of matrices n.
- Read the dimensions of the matrices in an array p.
  - Matrix A1 has dimensions p[0] × p[1].
  - Matrix A2 has dimensions p[1] × p[2].
  - Continue similarly for all matrices.
- Create a DP table m[n+1][n+1].
- Set m[i][i] = 0 because multiplying a single matrix requires no multiplication.
- Consider chain lengths from 2 to n.
- For every possible starting matrix i, calculate the ending matrix j.
- Try every possible split position k between i and j.
- Calculate the cost using:
  - cost = m[i][k] + m[k+1][j] + p[i-1] × p[k] × p[j]
- Store the minimum cost in m[i][j].
- The value m[1][n] gives the minimum number of scalar multiplications.
- Display the minimum multiplication cost.
- Stop.

Code:

```
# Matrix Chain Multiplication using Dynamic Programming

def matrix_chain_order(p, n):
    # m[i][j] stores minimum multiplication cost
    m = [[0 for j in range(n)] for i in range(n)]
```

```
    # Length of the chain
    for length in range(2, n):
        for i in range(1, n - length + 1):
            j = i + length - 1
            m[i][j] = float('inf')

            # Try all possible split positions
            for k in range(i, j):
                cost = (m[i][k] +
                        m[k + 1][j] +
                        p[i - 1] * p[k] * p[j])

                if cost < m[i][j]:
                    m[i][j] = cost

    return m[1][n - 1]


# Take input from the user
n = int(input("Enter the number of matrices: "))

print("Enter the dimensions:")

p = []

for i in range(n + 1):
    p.append(int(input()))

# Find minimum multiplication cost
minimum_cost = matrix_chain_order(p, n + 1)

# Display result
print("\nMinimum number of scalar multiplications:",
      minimum_cost)

# Enter your EN.No
print("EN.No:92460118706")
```

Output:

```
PS C:\Users\swamy> & 'C:\Users\swamy\AppData\Local\Programs\Python\Python315\python.exe' 'c:\Users\swamy\.vscode\ex
tensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '50711' '--' 'c:\Users\swamy\import tim
e.py'
Enter the number of matrices: 3
Enter the dimensions:
5
4
3
6

Minimum number of scalar multiplications: 150
EN.No:92460118706
```

Time Complexity:

Case	*Time Complexity*	*Description*
Best Case	*$O(n^3)$*	*All possible matrix split positions are checked.*
Average Case	*$O(n^3)$*	*Evaluates different splits.*
Worst case	*$O(n^3)$*	*Checks all possible splits.*

Space Complexity:

Space Complexity	*Description*
$O(n^2)$	*A 2D table is used to store the minimum multiplication costs for different matrix chains.*